



# SoC설계 및 프로그래밍



## Text LCD (FPGA Programming)

문 병 인

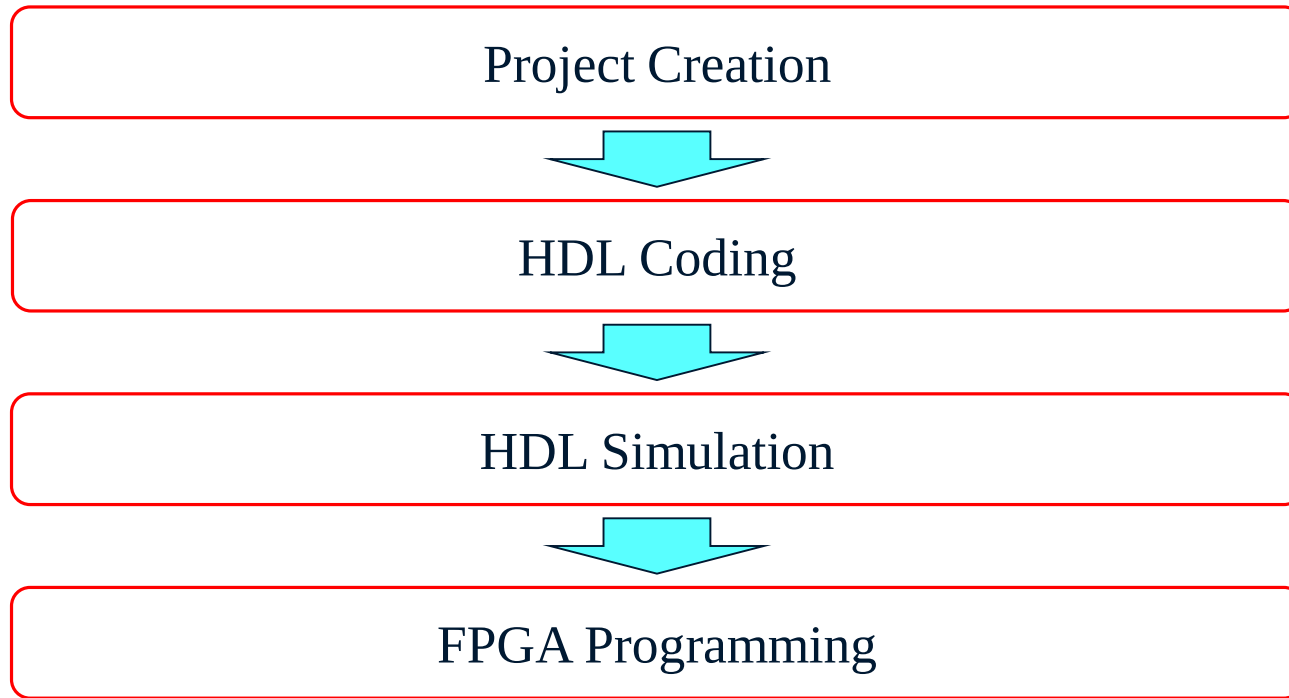
*bihmoon@knu.ac.kr*

경북대학교 전자공학부



경북대학교  
KYUNGPOOK  
NATIONAL UNIVERSITY

- 실습 과제 개요
- 실습 환경
- 실습 과정



## □ 실습 명

- ❖ Text LCD (FPGA Programming)

## □ 실습 목표

- ❖ Vivado 2019.1를 이용하여 ZYNQ의 PL에 text LCD 제어 회로 구현

## □ 실습내용

- ❖ HDL coding

- Verilog HDL을 이용하여 text LCD 제어 회로 구현

- ❖ HDL simulation

- ❖ FPGA(ZYNQ PL part) programming

- Pin assign
- Bitstream generation
- Programming

# 실습 환경 (1/3)

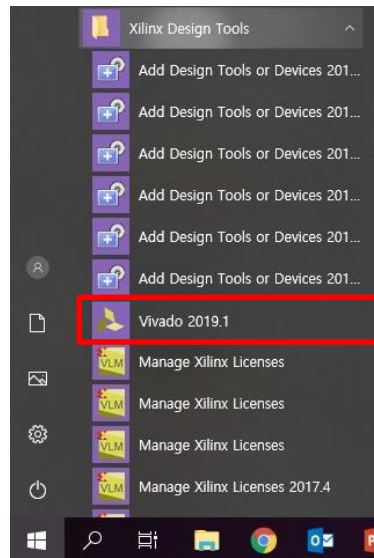
4

## □ Vivado 19.1

- ❖ Xilinx의 7 series FPGA 및 ZYNQ를 활용하기 위한 IDE(Integrated Development Environment) tool

## □ Vivado 19.1 설치 확인

- ❖ 시작 → 모든 프로그램 → Xilinx Design Tools → Vivado 2019.1 → Vivado 2019.1 또는 바탕화면에 있는 Vivado 2019.1 아이콘을 click



<바탕화면 아이콘>

# 실습 환경 (2/3)

5

## □ 실습 환경을 위한 directory 생성

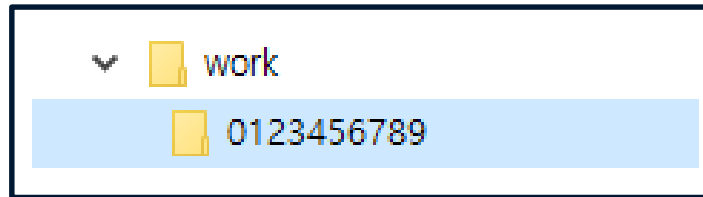
### ❖ Directory 생성 시 주의점

➤ 폴더 경로에 한글이 있는 경우 Vivado 에서 error 발생

### ❖ C 폴더 루트(c:\) 에 work 폴더를 생성

➤ work 폴더 내부에 학번을 이용한 폴더(ex. 0123456789) 를 생성

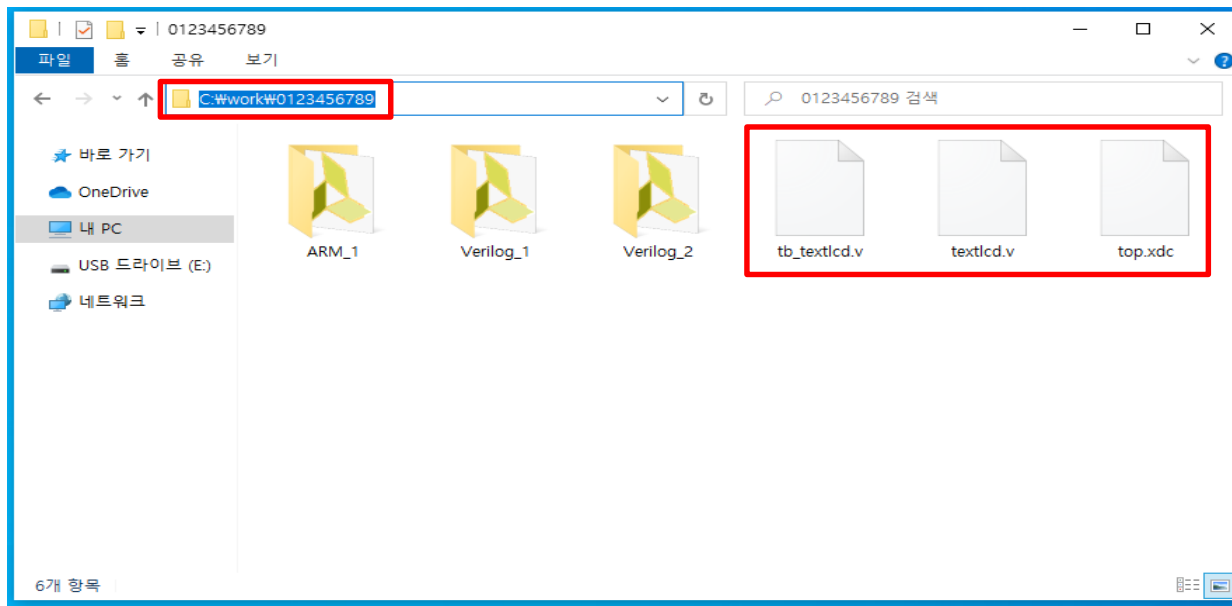
➤ 금일 실습을 수행할 폴더



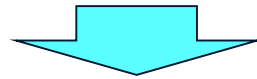
## □ 실습에서 사용할 file download

- ❖ 수업 게시판에서 verilog source file(**textlcd.v**) 및 testbench file(**tb\_textlcd.v**), constraint file(**top.xdc**)을 download 받은 후, 다음 경로에 위치

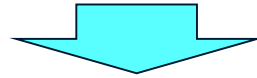
➤ “C:/work/“**학번**”/” 에 위치



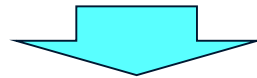
Project Creation



HDL Coding



HDL Simulation



FPGA Programming

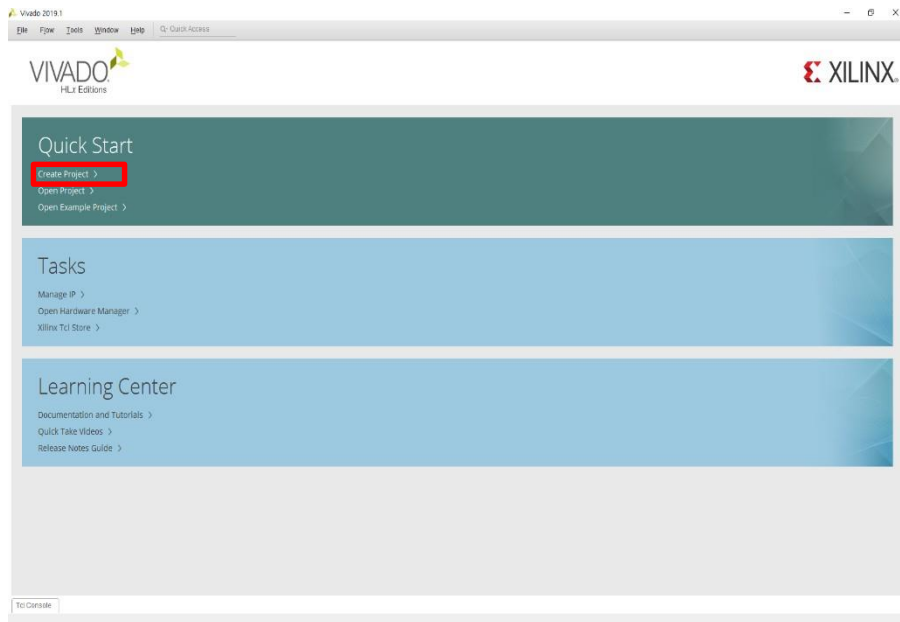
# Project Creation (1/8)

8

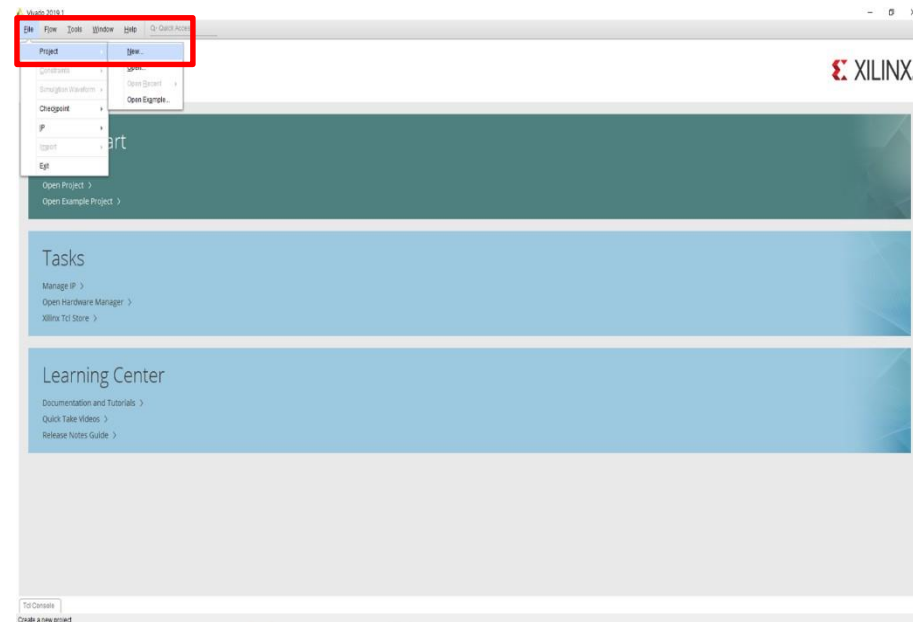
## □ 새 project 생성

❖ *Vivado는 project 단위의 개발 환경을 제공*

- Quick Start → Create Project 또는
- File → Project → New



or



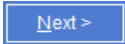


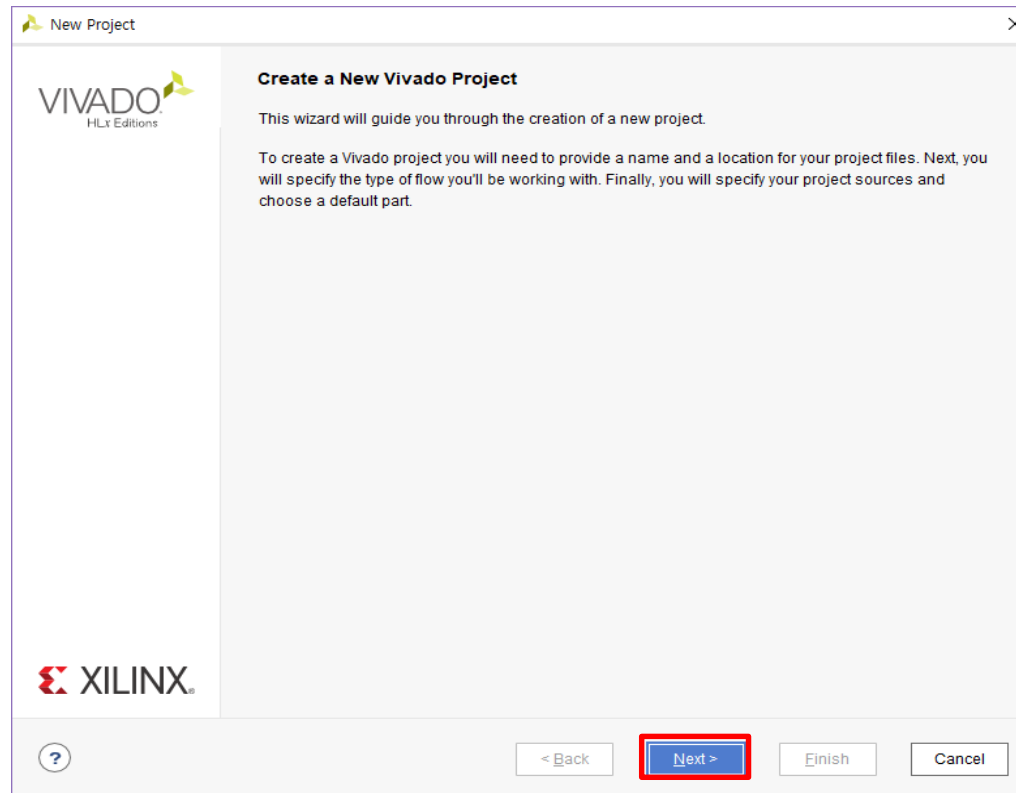
# Project Creation (2/8)

9

## □ New Project wizard

❖ *Project 생성에 관한 안내 문구와 함께 New Project wizard 실행*

➤  버튼을 클릭하여 New Project wizard 시작



# Project Creation (3/8)

10

## □ Project 경로 설정

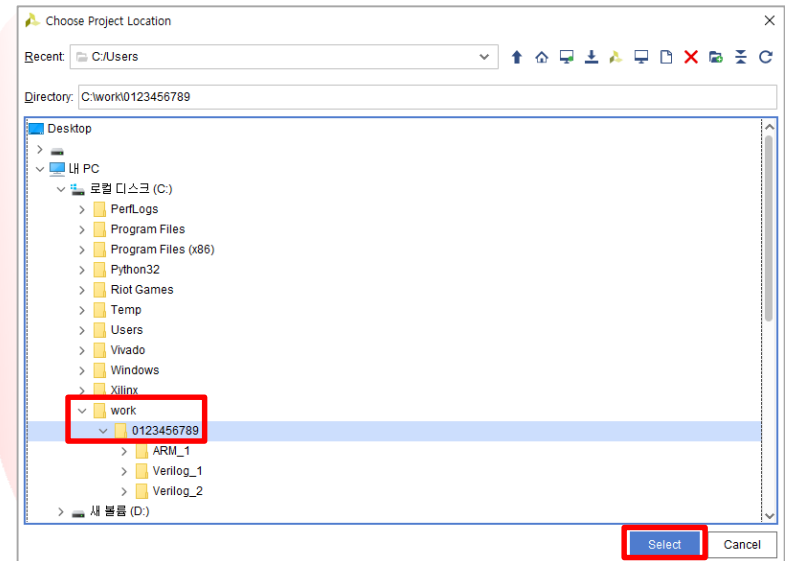
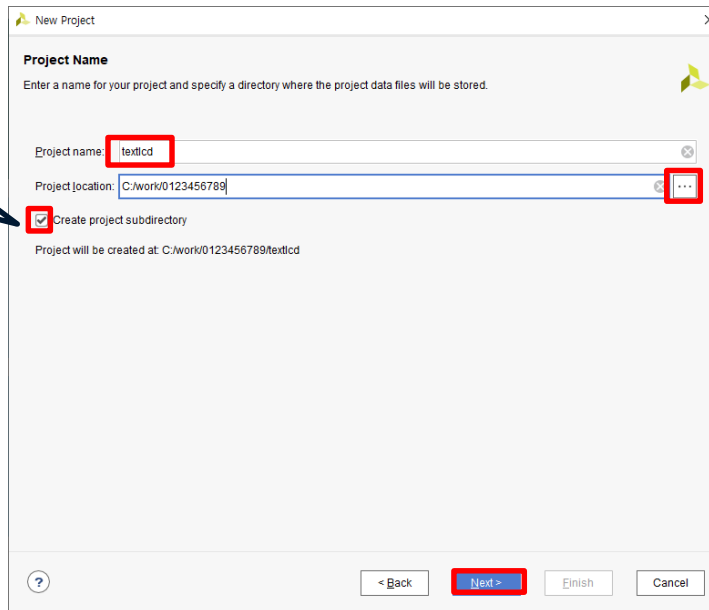
❖ 해당 project의 이름과 경로를 설정

➤ Project name : textlcd

➤ Project location : ... 버튼 click

▪ 컴퓨터 → 로컬 디스크 (C:) → work → “학번 폴더” 클릭 후, Select 클릭

➤ Create project subdirectory : project name으로 하위 폴더를 생성(check)



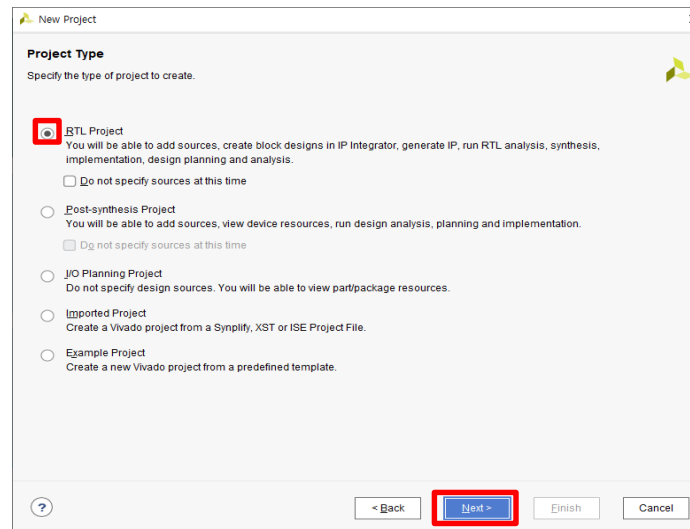
# Project Creation (4/8)

11

## □ Project Type 설정

### ❖ Project Type 을 설정 (Project Type 에 따라 개발 flow 가 달라짐)

- RTL Project는 HDL 코딩, synthesis, place and route, 등 FPGA와 ZYNQ 개발의 A to Z를 제공
- RTL Project 외의 Project Type은 다른 project의 결과를 불러와 활용
- Do not specify sources at this time : project 생성 시 소스 파일을 추가하지 않음(추후에 필요 시 추가 가능)



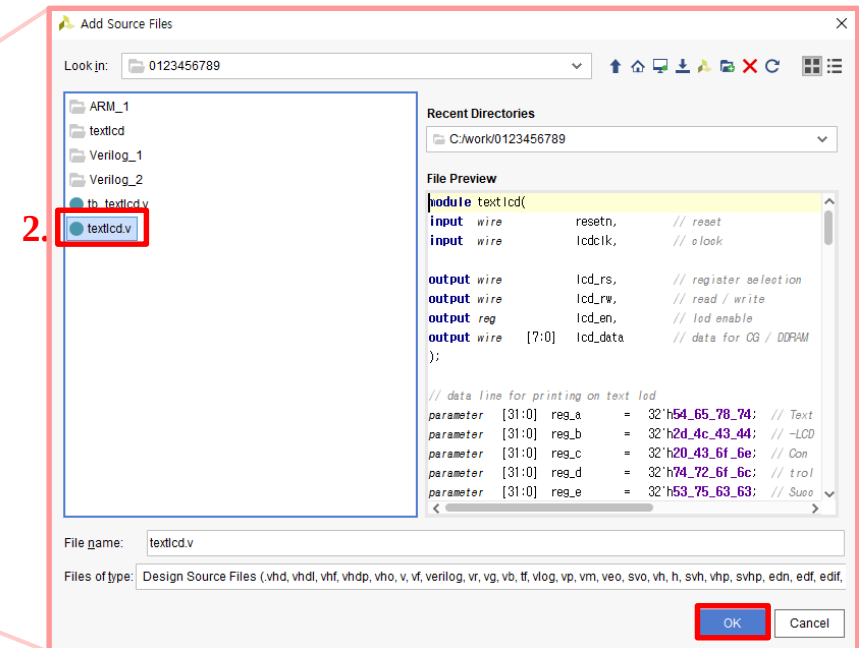
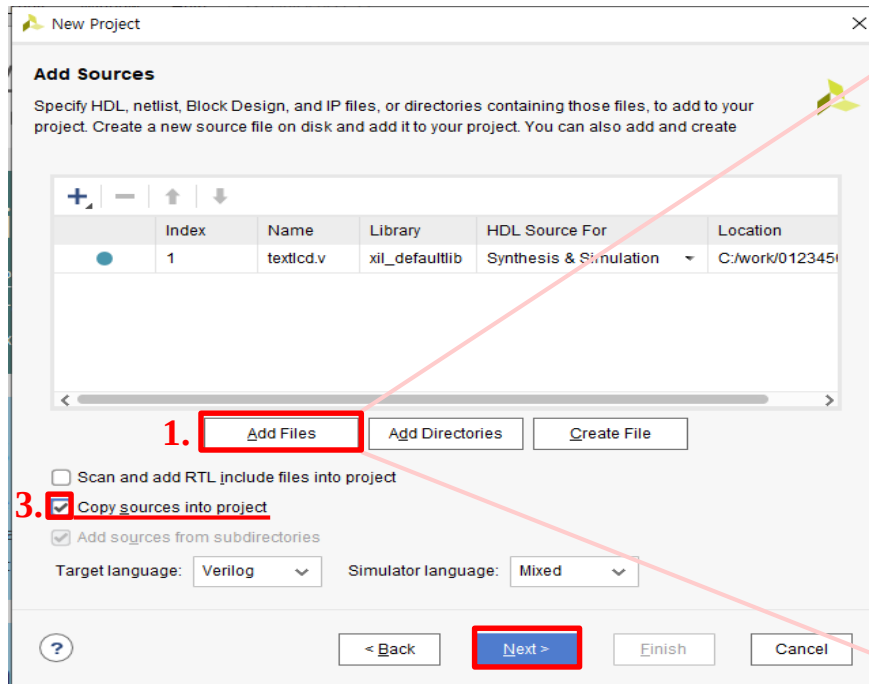
# Project Creation (5/8)

12

## □ Source file 추가

### ❖ HDL source file 생성 또는 추가

- 1. Add Files 클릭
- 2. 앞서 download한 verilog source file(textlcd.v) 선택 후 **OK** 클릭
- 3. Copy sources into project check 후 **Next >** 클릭



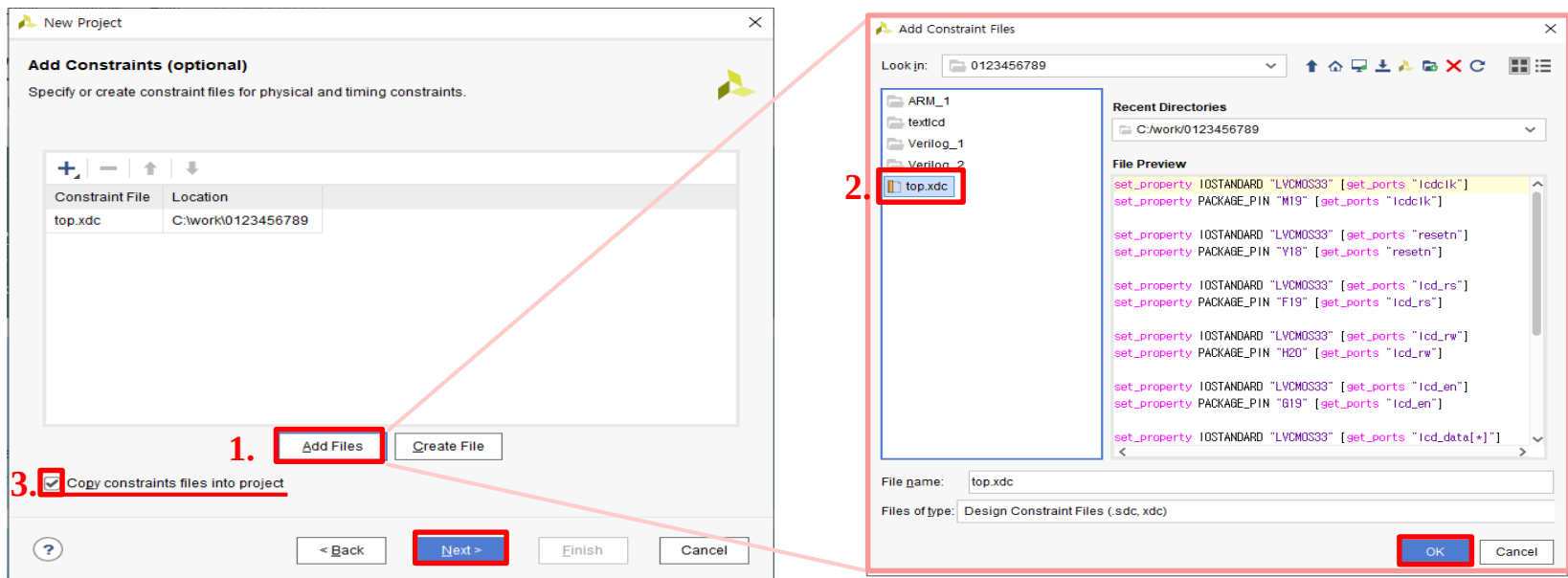
# Project Creation (6/8)

13

## □ Constraint file 추가

❖ *Constraint file 생성 또는 추가(constraint file은 pin assign, timing 에 관한 정보를 가짐)*

- 1. Add Files 클릭
- 2. 앞서 download한 constraint file(top.xdc) 선택 후 **OK** 클릭
- 3. Copy constraints files into project check 후 **Next >** 클릭



# Project Creation (7/8)

14

## □ Target device 설정

❖ 사용하는 FPGA / ZYNQ chip(part) 또는 board를 선택 (board의 경우 주변장치에 관한 preset 제공)

➤ Filter → Search에서 “xc7z020clg484-1”를 입력하여 해당 chip 선택

New Project

Default Part

Choose a default Xilinx part or board for your project.

Parts | Boards

Reset All Filters

Category: All Package: All Temperature: All

Family: All Speed: All Static power: All

Search: xc7z020clg484-1 (1 match)

| Part            | I/O Pin Count | Available IOBs | LUT Elements | FlipFlops | Block RAMs | Ultra RAMs | Datasheet |
|-----------------|---------------|----------------|--------------|-----------|------------|------------|-----------|
| xc7z020clg484-1 | 484           | 200            | 53200        | 106400    | 140        | 0          | 22        |

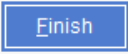
< Back Next > Finish Cancel

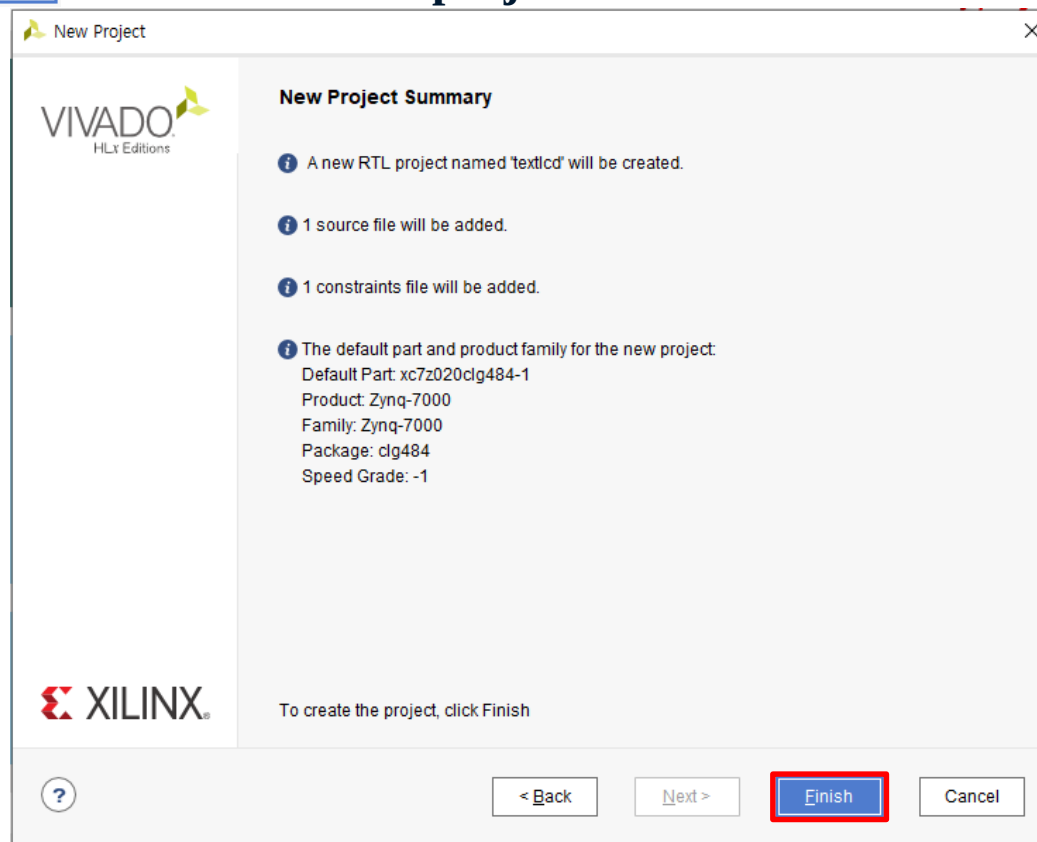
# Project Creation (8/8)

15

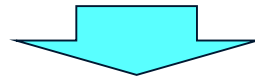
## □ Project 생성 마무리

❖ 앞서 설정한 옵션에 관한 Summary를 확인

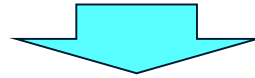
➤  버튼을 click하여 project 생성 마침



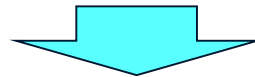
Project Creation



HDL Coding



HDL Simulation



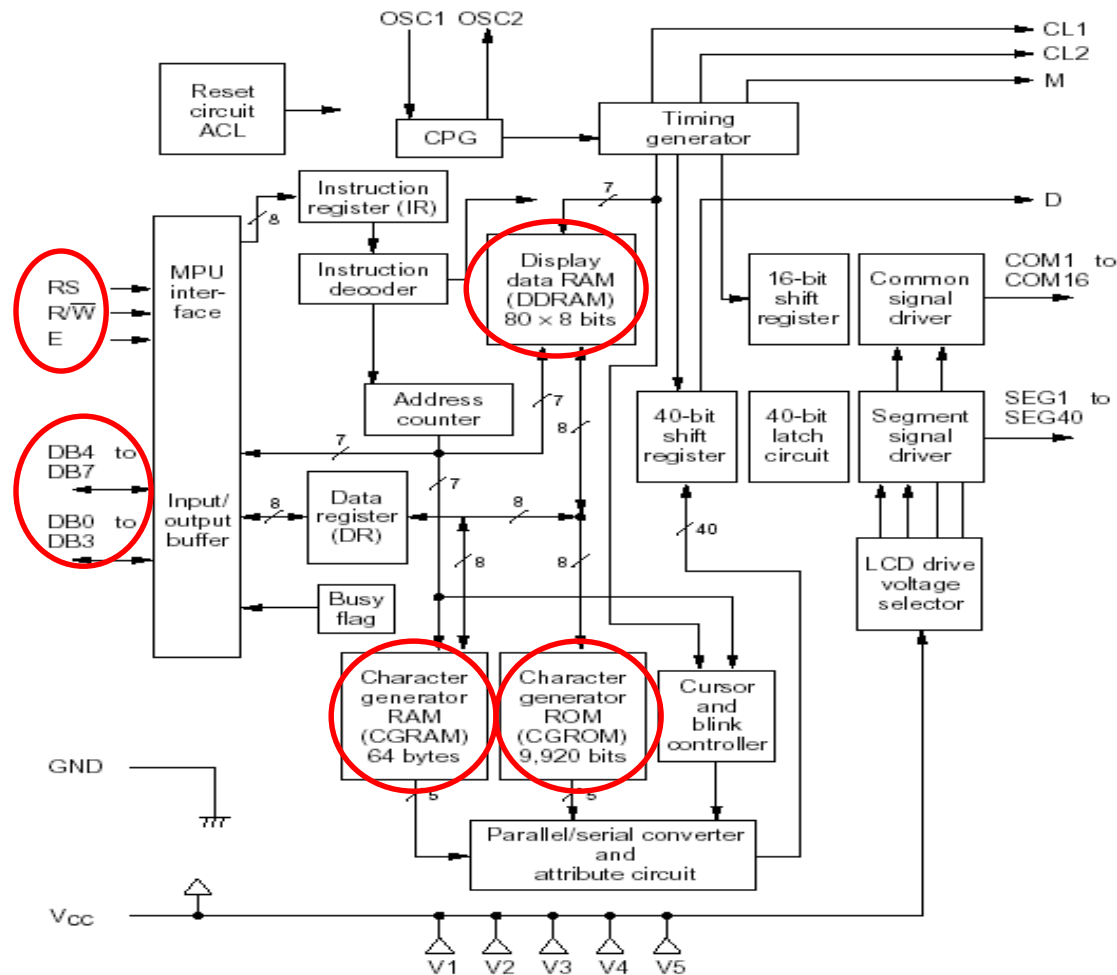
FPGA Programming



# Text LCD Module

17

## □ Text LCD controller / driver circuit



## □D.D.RAM(Display Data RAM)

- ❖ 8-bit 문자코드의 display data를 저장하는 RAM
- ❖ 최대 용량은  $80 \times 8\text{-bit}$ 로 80 문자 저장 가능
- ❖ 행별로 40 문자의 data를 저장

## □C.G.ROM(Character Generator ROM)

- ❖ ASCII 문자의 글씨체 정보를 저장하고 있는 ROM

## □C.G.RAM(Character Generator RAM)

- ❖ 사용자가 정의한 글씨체를 download 하는 RAM
- ❖ ASCII 문자 외의 사용자 정의 문자를 출력할 때 사용

## □ Text LCD 입력 및 제어 신호

### ❖ E(Enable)

- Text LCD에 제어 명령이나 data를 쓰거나 읽는 등의 동작을 하려면 이 pin이 high가 되어야 함

### ❖ RS(Register Select)

- 0 : 명령어 mode
- 1 : memory의 값을 읽거나 쓰는 mode

### ❖ R/W(Read/Write)

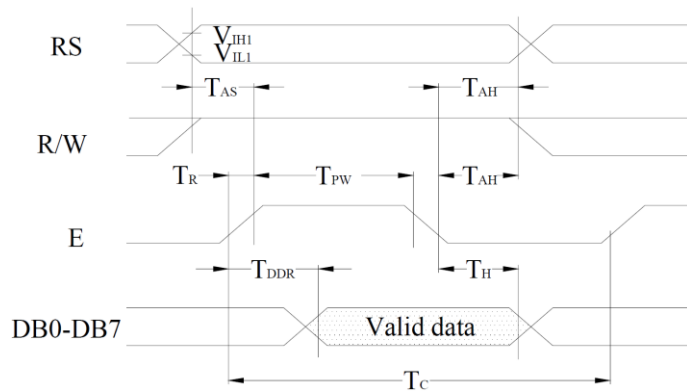
- 0 : data 쓰기
- 1 : data 읽기

### ❖ DB0~DB7

- Data를 주고받는 data bus

# Text LCD Timing Characteristics 20

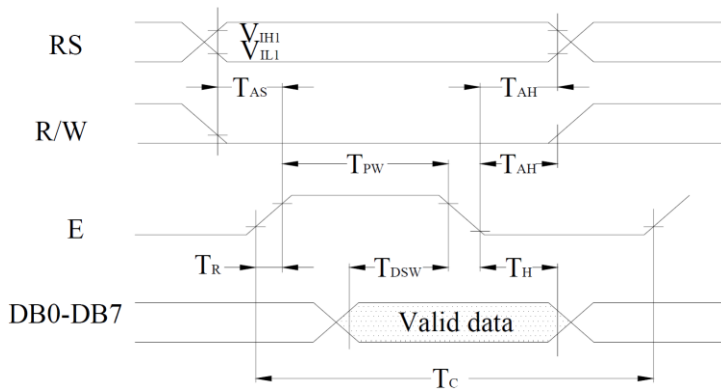
## □ Text LCD read / write timing



| Symbol     | Characteristics         | Test Condition  | Min. | Typ. | Max. | Unit |
|------------|-------------------------|-----------------|------|------|------|------|
| $T_C$      | Enable Cycle Time       | Pin E           | 1200 | -    | -    | ns   |
| $T_{PW}$   | Enable Pulse Width      | Pin E           | 140  | -    | -    | ns   |
| $T_R, T_F$ | Enable Rise / Fall Time | Pin E           | -    | -    | 25   | ns   |
| $T_{AS}$   | Address Setup Time      | Pins: RS, RW, E | 0    | -    | -    | ns   |
| $T_{AH}$   | Address Hold Time       | Pins: RS, RW, E | 10   | -    | -    | ns   |
| $T_{DDR}$  | Data Setup Time         | Pins: DB0~DB7   | -    | -    | 100  | ns   |
| $T_H$      | Data Hold Time          | Pins: DB0~DB7   | 10   | -    | -    | ns   |

( $V_{CC} = +5V, T_a = 25^\circ C$ )

(a) Reading data from Text LCD to FPGA



| Symbol     | Characteristics         | Test Condition  | Min. | Typ. | Max. | Unit |
|------------|-------------------------|-----------------|------|------|------|------|
| $T_C$      | Enable Cycle Time       | Pin E           | 1200 | -    | -    | ns   |
| $T_{PW}$   | Enable Pulse Width      | Pin E           | 140  | -    | -    | ns   |
| $T_R, T_F$ | Enable Rise / Fall Time | Pin E           | -    | -    | 25   | ns   |
| $T_{AS}$   | Address Setup Time      | Pins: RS, RW, E | 0    | -    | -    | ns   |
| $T_{AH}$   | Address Hold Time       | Pins: RS, RW, E | 10   | -    | -    | ns   |
| $T_{DSW}$  | Data Setup Time         | Pins: DB0~DB7   | 40   | -    | -    | ns   |
| $T_H$      | Data Hold Time          | Pins: DB0~DB7   | 10   | -    | -    | ns   |

( $V_{CC} = +5V, T_a = 25^\circ C$ )

(b) Writing data from FPGA to Text LCD

# Text LCD Instructions (1/4)

21

## □ Text LCD instruction table

| Instructions               | RS | R/W | D7      | D6  | D5  | D4 | D3  | D2  | D1  | D0 |
|----------------------------|----|-----|---------|-----|-----|----|-----|-----|-----|----|
| Clear display              | 0  | 0   | 0       | 0   | 0   | 0  | 0   | 0   | 0   | 1  |
| Return home                | 0  | 0   | 0       | 0   | 0   | 0  | 0   | 0   | 1   | X  |
| Entry mode set             | 0  | 0   | 0       | 0   | 0   | 0  | 0   | 1   | I/D | S  |
| Display ON/OFF             | 0  | 0   | 0       | 0   | 0   | 0  | 1   | D   | C   | B  |
| Cursor or display shift    | 0  | 0   | 0       | 0   | 0   | 1  | S/C | R/L | X   | X  |
| Function set               | 0  | 0   | 0       | 0   | 1   | DL | N   | F   | X   | X  |
| Set C.G.RAM address        | 0  | 0   | 0       | 1   | ACG |    |     |     |     |    |
| Set D.D.RAM address        | 0  | 0   | 1       | ADD |     |    |     |     |     |    |
| Read busy flag and address | 0  | 1   | BF      | AC  |     |    |     |     |     |    |
| Write data to RAM          | 1  | 0   | 출력 data |     |     |    |     |     |     |    |
| Read data from RAM         | 1  | 1   | 입력 data |     |     |    |     |     |     |    |

## □ Text LCD instruction table 설명

### ❖ Clear display

- D.D.RAM의 모든 address에 '20H'(space) write(빈 화면을 만듦)
- D.D.RAM의 0번지가 set
- Display 원위치(shift 되어 있다면)

### ❖ Return home

- Cursor의 위치(D.D.RAM address)가 0번지로 돌아감(1행의 첫 번째 열)
- Display 원위치(shift 되어 있다면)

### ❖ Entry mode set

- Cursor의 진행방향 표시를 shift 시킬 것인가를 결정
  - I/D = 1 : D.D.RAM에 read/write할 때 address를 1 증가
  - I/D = 0 : D.D.RAM에 read/write할 때 address를 1 감소
  - S = 1 : 화면의 글자가 shift 됨(I/D = 0 이면 right, I/D = 1 이면 left)
  - S = 0 : 화면의 글자가 shift 되지 않음

# Text LCD Instructions (3/4)

23

## ❖ Display ON / OFF

- $D = 1$  : display on,       $D = 0$  : display off
- $C = 1$  : cursor on,       $C = 0$  : cursor off
- $B = 1$  : cursor 위치의 문자가 깜박임

## ❖ Cursor or display shift

- $S/C = 1$  : display shift,       $S/C = 0$  : cursor shift
- $R/L = 1$  : right shift,       $R/L = 0$  : left shift

## ❖ Function set

- $DL = 1$  : 8-bit data line,       $DL = 0$  : 4-bit data line
- $N = 1$  : 2행,       $N = 0$  : 1행
- $F = 1$  : font size가 5 \* 10 dots,       $F = 0$  : 5 \* 8 dots

## ❖ Set C.G.RAM / D.D.RAM address

- C.G.RAM / D.D.RAM의 address를 ACG(address) bits로 setting
- 이후의 read / write는 각 설정 RAM에 관한 것이 됨
- D.D.RAM의 경우 1행은 00H~27H, 2행은 28H~4FH로 할당

# Text LCD Instructions (4/4)

24

## ❖ Read busy flag and address

- 전에 받은 instruction에 의해서 LCD가 내부 동작 중인지를 알기 위해서 busy flag 신호를 읽음
- Busy flag 신호를 읽음과 동시에 2진수로 표시되는 address counter의 값을 읽음

## ❖ Write data to RAM

- C.G.RAM / D.D.RAM address set 되어 있는 address로 data write
- Write 후 address 및 display shift는 entry mode 결정에 따름

## ❖ Read data from RAM

- C.G.RAM / D.D.RAM address set 되어 있는 address에서 data read



## □ Text LCD 초기 동작 sequence

1. 전원 공급
2. Function set 명령(001xxx00)을 보냄
3. Display ON / OFF control 명령(00001xxx)을 보냄
4. Entry mode set 명령(000001xx)을 보냄
  - \* Return home 명령(0000001x)과 clear display 명령(00000001)을 추가 가능
5. D.D.RAM address 설정 명령(1xxxxxxxxx)을 보냄
6. Data를 D.D.RAM에 write
7. 필요에 따라 D.D.RAM address를 바꾸어 2행에 출력 가능

# Text LCD IP Design

26

□HDL을 이용하여 다음의 text를 Text LCD에 표시하는 IP 설계

Text-LCD Control  
Success SoC Lab



# HDL Coding (1/8)

27

## □ textlcd.v coding

❖ Project Manager → Sources → Hierarchy → Design Sources에서  
textlcd(textlcd.v) double click하여 source file open

❖ Source file open 후 coding

The screenshot shows the Vivado 2019.1 IDE. The top menu bar includes File, Edit, Flow, Tools, Reports, Window, Layout, View, and Help. Below the menu is a toolbar with various icons. The left sidebar contains the Flow Navigator and the PROJECT MANAGER. The PROJECT MANAGER pane shows the Sources tab with a list of Design Sources (1). The source file 'textlcd (textlcd.v)' is highlighted with a red box. The right pane shows the textlcd.v source file with the following code:

```
1 module textlcd(  
2   input wire      resetn,    // reset  
3   input wire      lcdclk,    // clock  
4  
5   output wire      lcd_rs,    // register selection  
6   output wire      lcd_rw,    // read / write  
7   output reg       lcd_en,    // lcd enable  
8   output wire [7:0] lcd_data, // data for CG / DDRAM  
9 );  
10  
11 // data line for printing on text lcd
```

The word 'Coding' is written in large red text over the code.

## □ Enable signal coding

❖ *Text LCD read / write timing 에서 보았듯이 enable signal(lcd\_en register)의 제어가 필요*

- count\_lcdclk == 11'd200 이면 lcd\_en set
- count\_lcdclk == 11'd1800 이면 lcd\_en reset

```
39 : // enable signal
40 : always@(posedge lcdclk or negedge resetn)
41 : begin
42 :     if(resetn == 1'b0) begin
43 :         lcd_en <= 1'b0;
44 :     end
45 :     else begin
46 :         //*****
47 :         //
48 :         //
49 :         //
50 :         // Coding here!
51 :         //
52 :         //
53 :         //
54 :         //*****
55 :     end
56 : end
```

Coding

## □ Clock counter coding

- ❖ 입력 *clock(lcdclk)* 2000 cycles 를 기준으로 1 개의 *text LCD instruction* 을 가함(*lcdclk 2000 cycles* 가 *text LCD* 의 한 동작 단위)
- *lcdclk* 를 이용하여 *count\_lcdclk* register 를 0 에서 1999 까지 count 하는 counter 설계

```
58 : // clock counter
59 @ always@(posedge lcdclk or negedge resetn)
60 begin
61     if(resetn == 1'b0) begin
62         count_lcdclk <= 11'd0;
63     end
64     else begin
65         //*****
66         //
67         // Coding here!
68         //
69         //
70         //*****
71     end
72 end
```

Coding

## □ Mode state counter coding

### ❖ *Text LCD의 mode state를 가리키는 counter 설계*

- count\_lcdclk를 이용하여 lcdclk 2000 cycles(count\_lcdclk == 11'd1999)마다 count\_mode register를 0에서 40까지 count하는 counter 설계 (단, reset 이후 첫 count 시에만 0에서 40을 count하고, 이후부터는 7에서 40까지 count)

```
74 : // mode state counter
75 : always@(posedge lcdclk or negedge resetn)
76 : begin
77 :     if(resetn == 1'b0) begin
78 :         count_mode <= 6'd0;
79 :     end
80 :     else begin
81 :         //*****
82 :         //
83 :         //
84 :         //
85 :         //
86 :         // Coding here!
87 :         //
88 :         //
89 :         //
90 :         //
91 :         //*****
92 :     end
93 : end
```

Coding

## □ Mode state coding

❖ *Text LCD 초기 동작 sequence에 따라 mode state 설계*

➤ count\_mode에 따라 lcd\_mode 값 설정

```
95 // mode state
96 always@(posedge lcdclk or negedge resetn)
97 begin
98     if(resetn == 1'b0) begin
99         lcd_mode <= mode_pwron;
100     end
101     else begin
102         // mode change
103         case(count_mode)
104             6'd0 : lcd_mode <= mode_pwron;
105             6'd1 : lcd_mode <= mode_fnset;
106             6'd2 : lcd_mode <= mode_onoff;
107             6'd3 : lcd_mode <= mode_entr1;
108             6'd4 : lcd_mode <= mode_entr2;
109             6'd5 : lcd_mode <= mode_entr3;
110             6'd6 : lcd_mode <= mode_setal;
111             6'd7 : lcd_mode <= mode_wrlst;
112             6'd23 : lcd_mode <= mode_setal;
113             6'd24 : lcd_mode <= mode_wr2nd;
114             6'd40 : lcd_mode <= mode_delay;
115             default : lcd_mode <= lcd_mode;
116         endcase
117     end
118 end
```

Typing

## □ Decoder coding

### ❖ *Text LCD*를 구동하기 위한 초기설정

- Display ON / OFF 설정(D = 1, C = 1)
- Entry mode 설정(I/D = 1, S = 0)
- Return home 설정
- Clear display
- D.D.RAM address 설정 (ADD = 00000000)

Function set (D/L = 1, N = 1, F = 0)

```
128 case(lcd_mode)
129     mode_pwron : set_data = {2'b00, 8'h38};
130     mode_fnset  : set_data = {2'b00, 8'h38};
131     mode_onoff  : // Coding here!
132     mode_entr1  : // Coding here!
133     mode_entr2  : // Coding here!
134     mode_entr3  : // Coding here!
135     mode_setal  : // Coding here!
136     mode_wr1st  :
```

Coding  
(위 순서대로)



# HDL Coding (7/8)

33

## □ Decoder coding(Cont.)

❖ Data를 1행과 2행에 출력하기 위해 D.D.RAM에 write 하는 case 구문

Typing

```
136 mode_wr1st :  
137 begin  
138     case(count_mode)  
139         6'd7 : set_data = {1'b1, 1'b0, reg_a[31:24]};  
140         6'd8 : set_data = {1'b1, 1'b0, reg_a[23:16]};  
141         6'd9 : set_data = {1'b1, 1'b0, reg_a[15: 8]};  
142         6'd10 : set_data = {1'b1, 1'b0, reg_a[7 : 0]};  
143         6'd11 : set_data = {1'b1, 1'b0, reg_b[31:24]};  
144         6'd12 : set_data = {1'b1, 1'b0, reg_b[23:16]};  
145         6'd13 : set_data = {1'b1, 1'b0, reg_b[15: 8]};  
146         6'd14 : set_data = {1'b1, 1'b0, reg_b[7 : 0]};  
147         6'd15 : set_data = {1'b1, 1'b0, reg_c[31:24]};  
148         6'd16 : set_data = {1'b1, 1'b0, reg_c[23:16]};  
149         6'd17 : set_data = {1'b1, 1'b0, reg_c[15: 8]};  
150         6'd18 : set_data = {1'b1, 1'b0, reg_c[7 : 0]};  
151         6'd19 : set_data = {1'b1, 1'b0, reg_d[31:24]};  
152         6'd20 : set_data = {1'b1, 1'b0, reg_d[23:16]};  
153         6'd21 : set_data = {1'b1, 1'b0, reg_d[15: 8]};  
154         default : set_data = {1'b1, 1'b0, reg_d[7 : 0]};  
155     endcase  
156 end
```

Typing

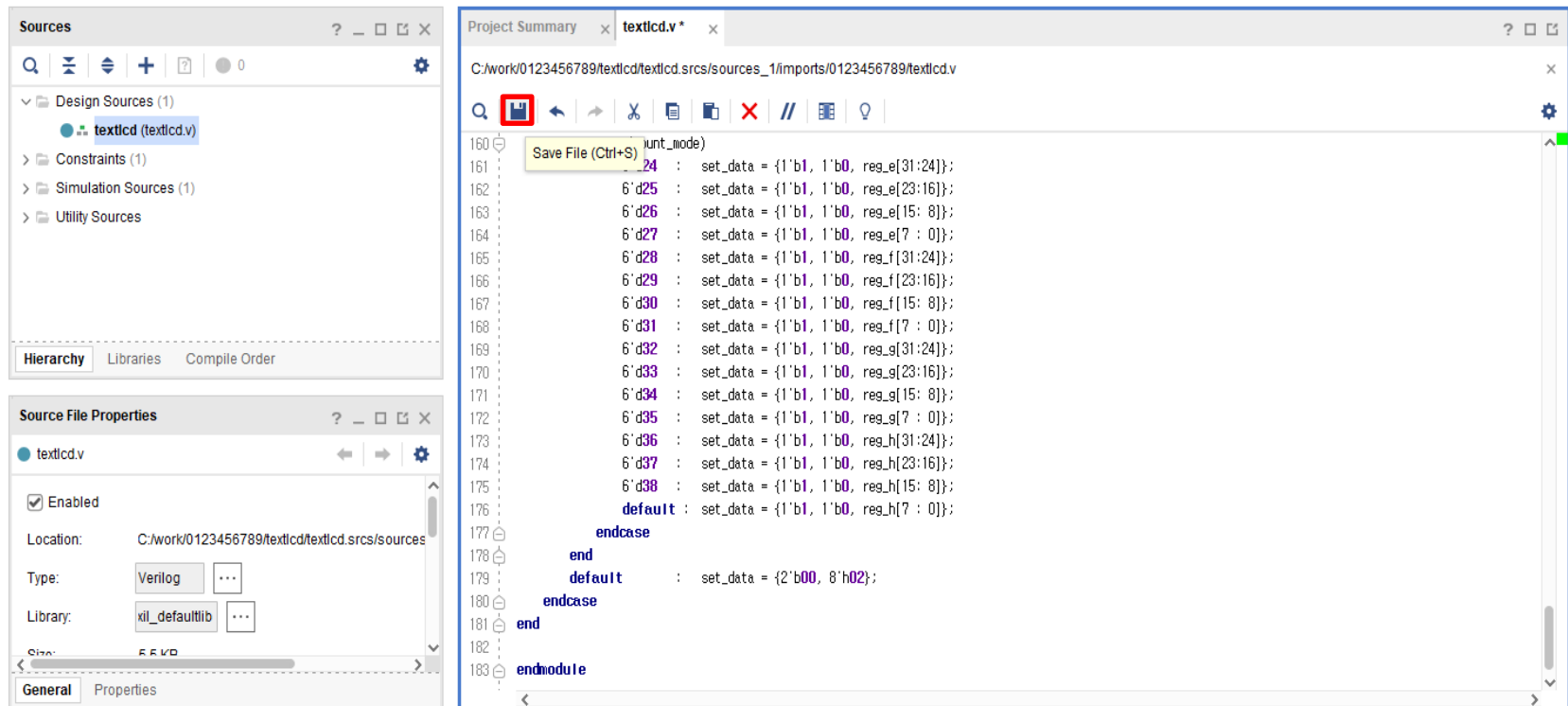
```
158 mode_wr2nd :  
159 begin  
160     case(count_mode)  
161         6'd24 : set_data = {1'b1, 1'b0, reg_e[31:24]};  
162         6'd25 : set_data = {1'b1, 1'b0, reg_e[23:16]};  
163         6'd26 : set_data = {1'b1, 1'b0, reg_e[15: 8]};  
164         6'd27 : set_data = {1'b1, 1'b0, reg_e[7 : 0]};  
165         6'd28 : set_data = {1'b1, 1'b0, reg_f[31:24]};  
166         6'd29 : set_data = {1'b1, 1'b0, reg_f[23:16]};  
167         6'd30 : set_data = {1'b1, 1'b0, reg_f[15: 8]};  
168         6'd31 : set_data = {1'b1, 1'b0, reg_f[7 : 0]};  
169         6'd32 : set_data = {1'b1, 1'b0, reg_g[31:24]};  
170         6'd33 : set_data = {1'b1, 1'b0, reg_g[23:16]};  
171         6'd34 : set_data = {1'b1, 1'b0, reg_g[15: 8]};  
172         6'd35 : set_data = {1'b1, 1'b0, reg_g[7 : 0]};  
173         6'd36 : set_data = {1'b1, 1'b0, reg_h[31:24]};  
174         6'd37 : set_data = {1'b1, 1'b0, reg_h[23:16]};  
175         6'd38 : set_data = {1'b1, 1'b0, reg_h[15: 8]};  
176         default : set_data = {1'b1, 1'b0, reg_h[7 : 0]};  
177     endcase  
178 end
```

# HDL Coding (8/8)

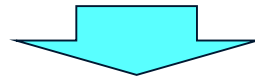
34

## □ textlcd.v coding 완료 후 저장

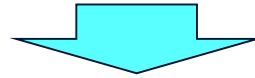
- ❖ Source window toolbar에서 Save File(  ) click 또는
- ❖ 단축키 Ctrl + S



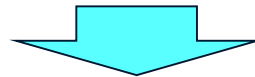
Project Creation



HDL Coding



HDL Simulation



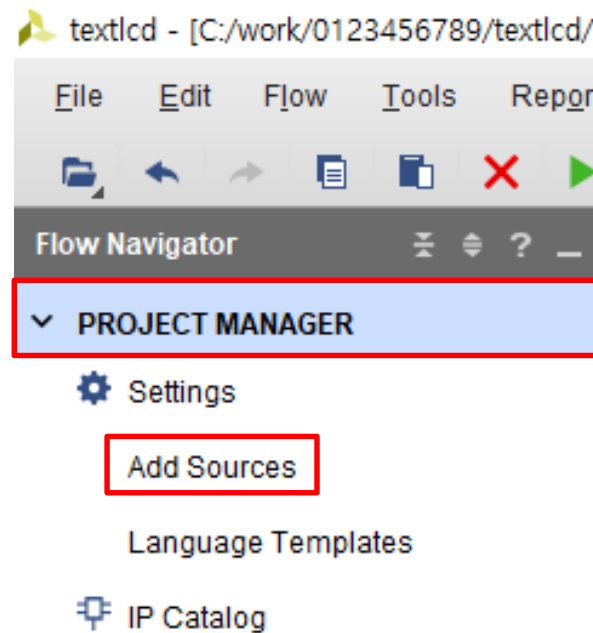
FPGA Programming

# Testbench 불러오기

36

## □ Simulation을 위한 testbench file 불러오기

❖ Project Manager → Add Sources 클릭

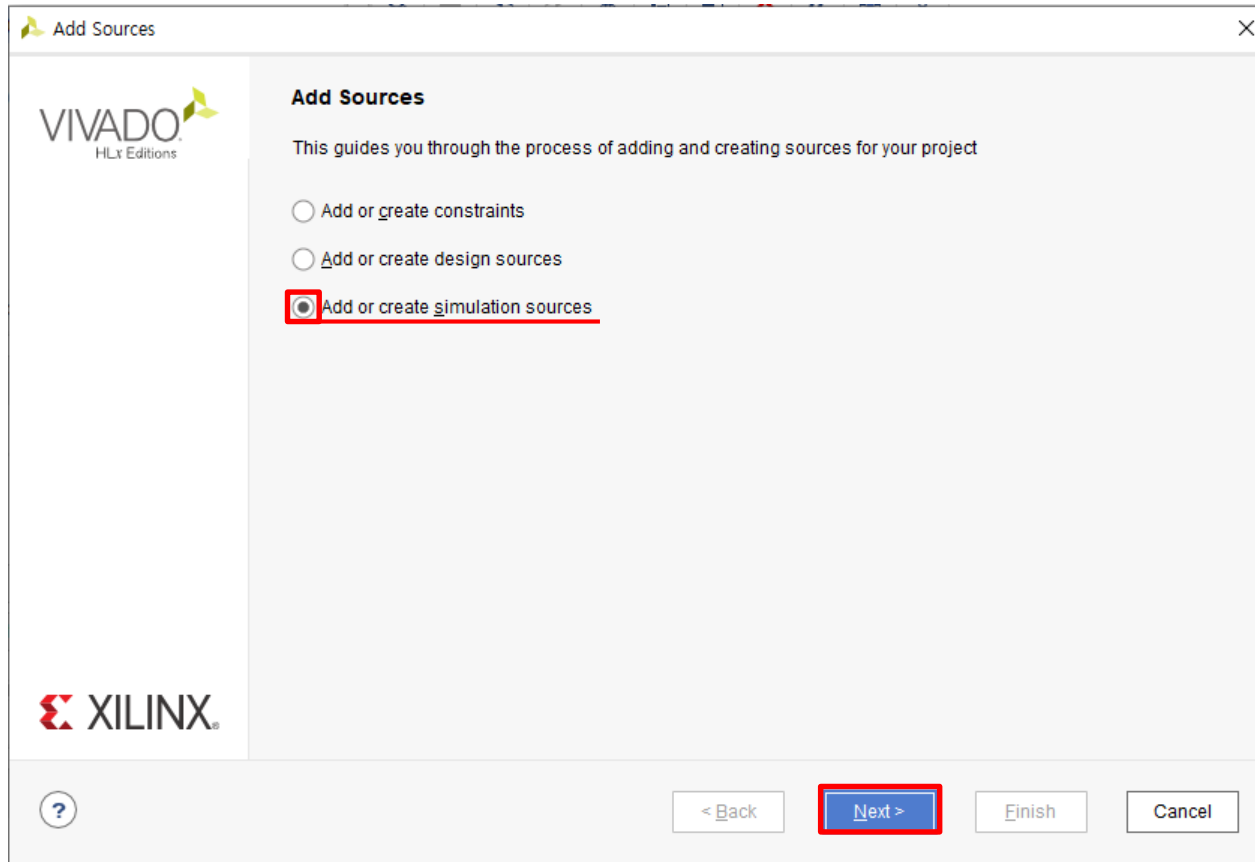


# Add Sources Wizard (1/2)

37

□ Source file을 불러오기 위한 wizard 실행

❖ Add or create simulation sources 체크 → **Next >** 클릭

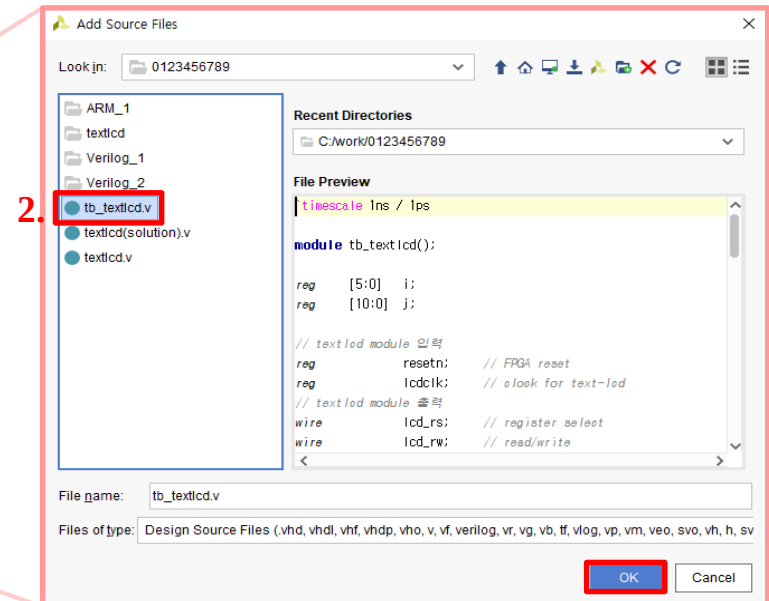
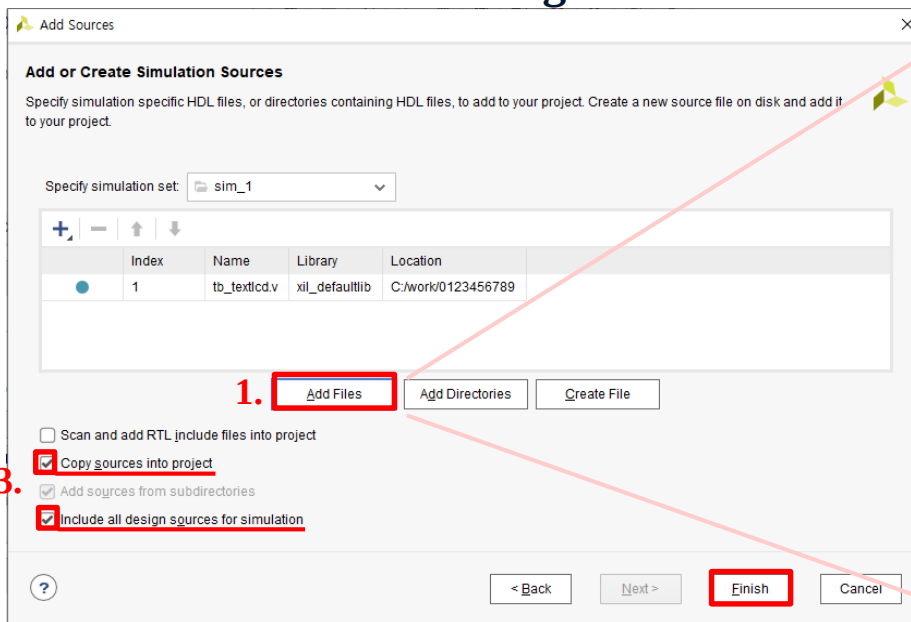


# Add Sources Wizard (2/2)

38

## □ Simulation source file 추가

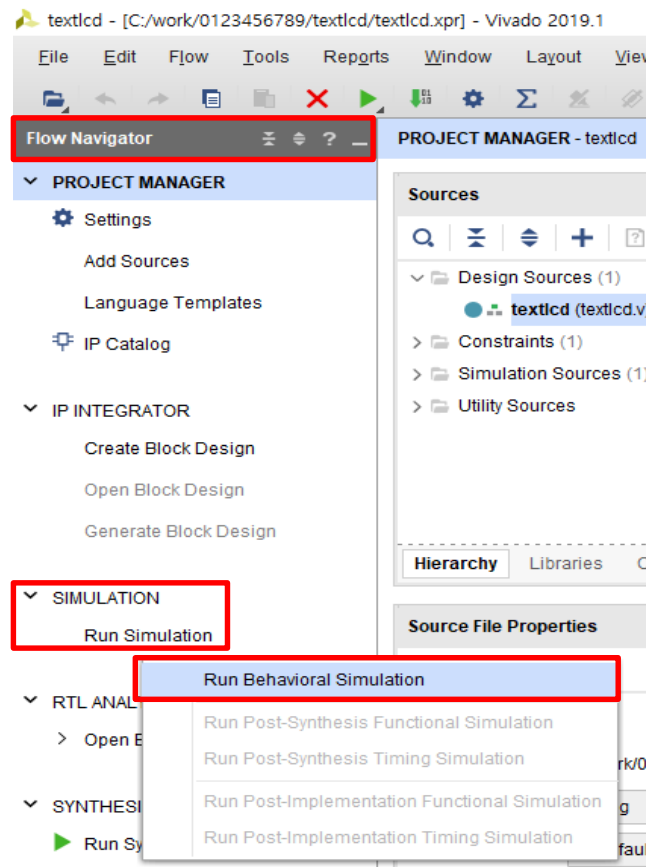
- ❖ 1. Add Files 클릭
- ❖ 2. 앞서 download한 testbench file(**tb\_textlcd.v**) 선택 후 **OK** 클릭
- ❖ 3. Check 확인 후 Finish click
  - Copy sources into project
  - Include all design sources for simulation



# HDL Simulation (1/6)

39

□ Flow Navigator → Simulation → Run Simulation 클릭 →  
Run Behavioral Simulation 클릭

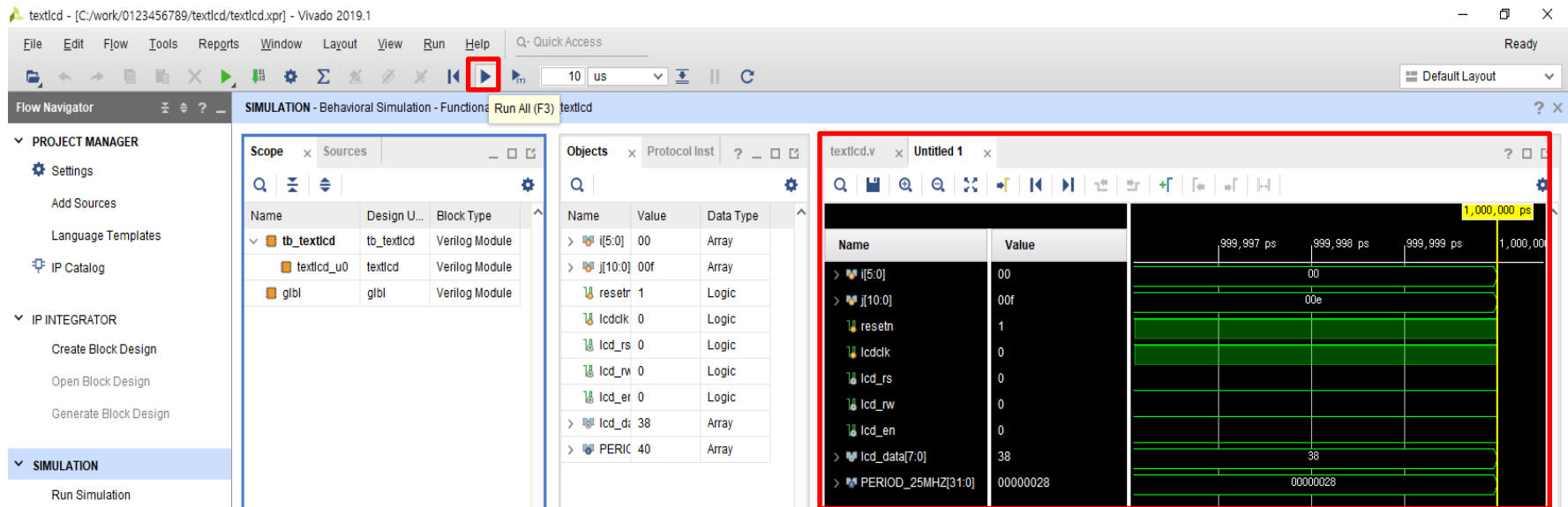


# HDL Simulation (2/6)

40

## □ Behavioral Simulation window 안에 waveform window 생성

- ❖ Simulation top인 testbench의 signals이 waveform window에 자동 추가
- ❖ Simulation을 위해 Run All(▶) 클릭 또는 단축키 F3



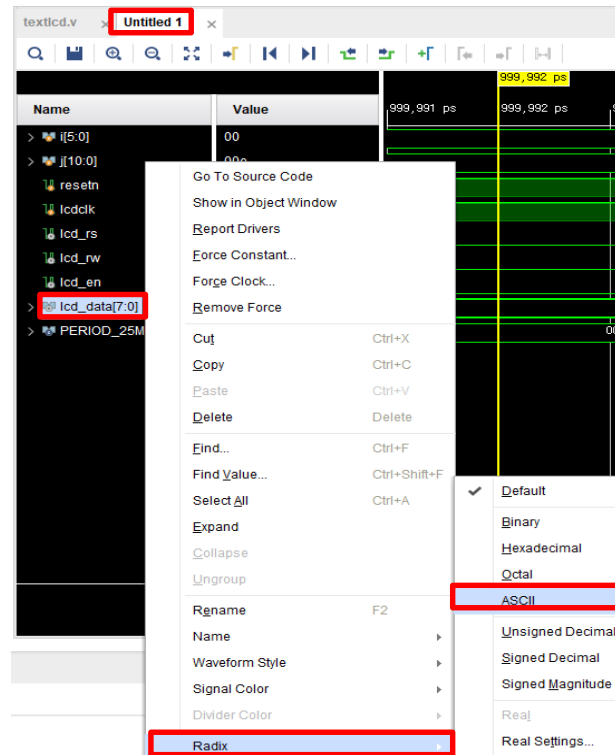


# HDL Simulation (3/6)

41

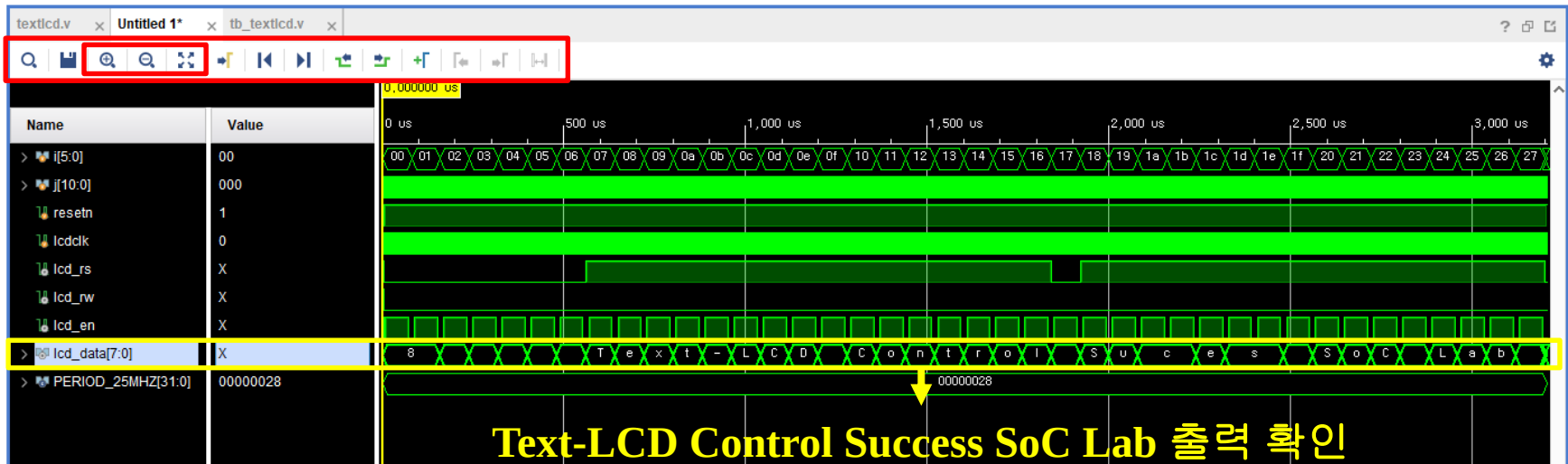
## □ Waveform의 binary 값을 ASCII로 변환

- ❖ lcd\_data[7:0]의 radix를 binary에서 ASCII로 변환
- ❖ Waveform window에서 해당 signal(lcd\_data[7:0]) 클릭 → 마우스 오른쪽 클릭 → Radix → ASCII 클릭



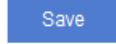
## □ Simulation 결과 확인

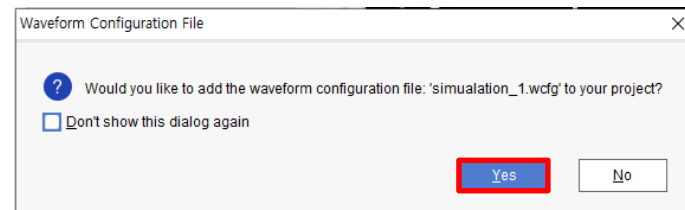
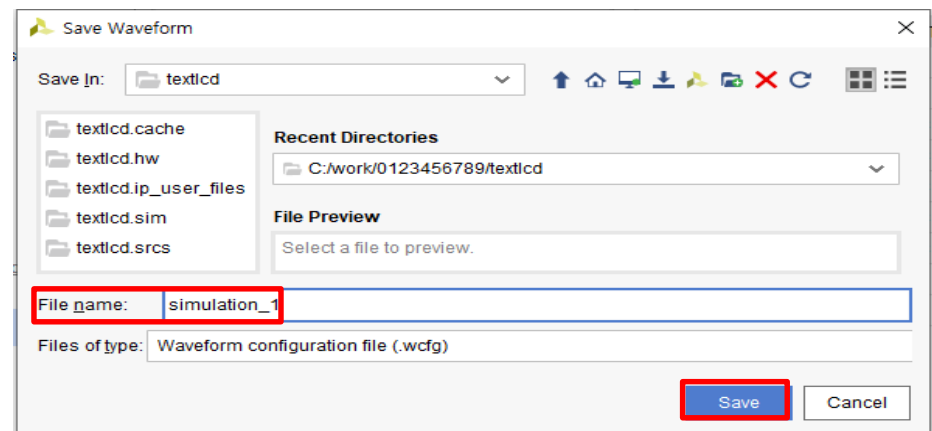
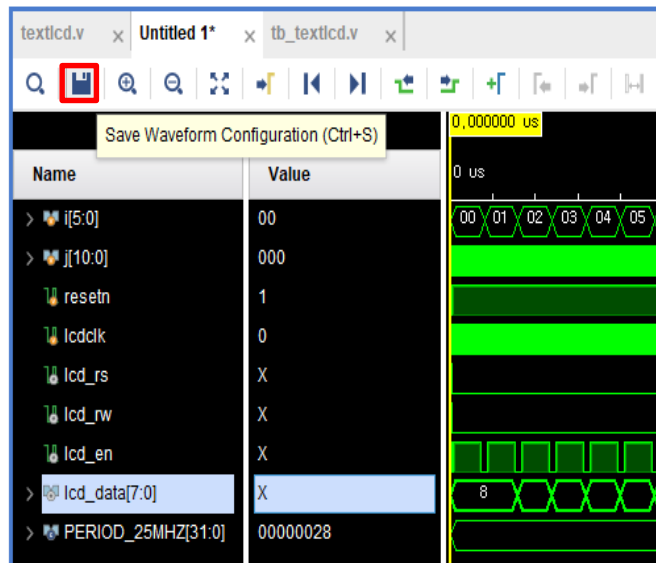
- ❖ Waveform window 좌측의 toolbar로 waveform을 조절
  - Zoom Fit(  ) : waveform을 window 크기에 맞게끔 zoom
  - Zoom In(  ) : 현재 cursor 위치에서 waveform 확대
  - Zoom Out(  ) : 현재 cursor 위치에서 waveform 축소
- ❖ Zoom Fit 아이콘(  )을 클릭하여 올바른 waveform이 출력되는지 확인
- ❖ 만약 올바른 waveform이 출력되지 않으면 코드를 수정한 후 다시 simulation을 실행



## □ Waveform configuration 저장

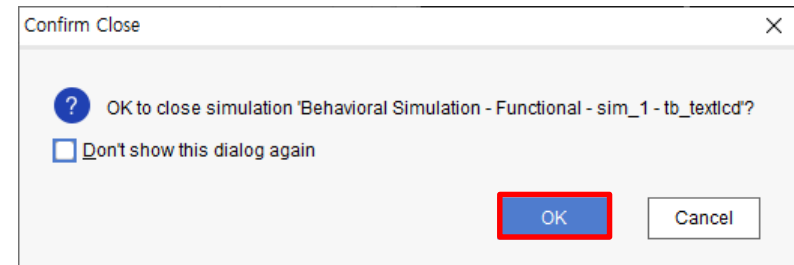
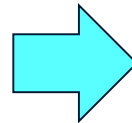
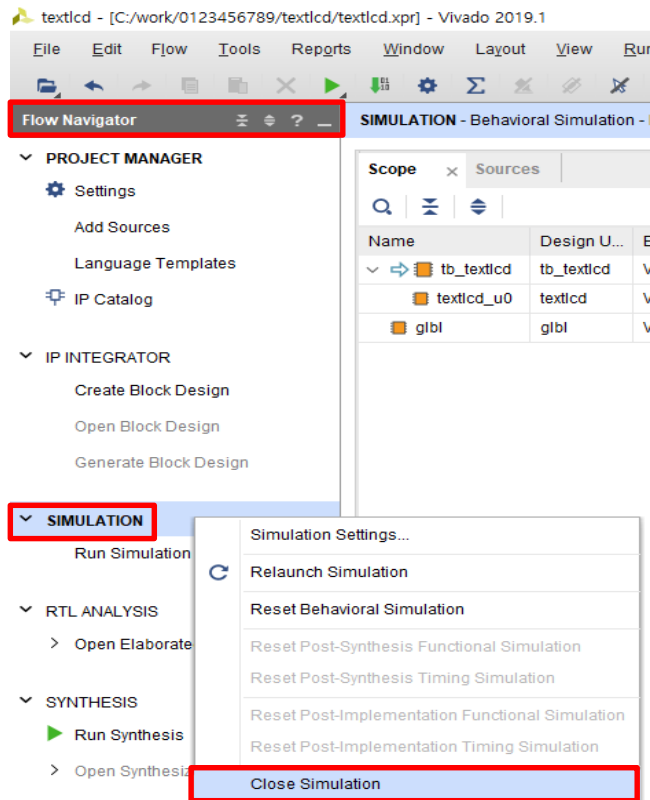
❖ *Waveform 에 추가한 signals와 signal color, radix 등의 configuration 정보 저장(해당 project 경로에 저장)*

- Waveform window toolbar → Save Waveform Configuration(  ) 클릭
- File name : simulation\_1 입력 후  클릭

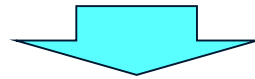


## □ Simulation 종료하기

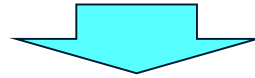
❖ Flow Navigator → Simulation 마우스 오른쪽 클릭 → Close  
Simulation 클릭



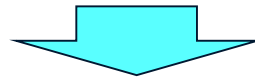
Project Creation



HDL Coding



HDL Simulation



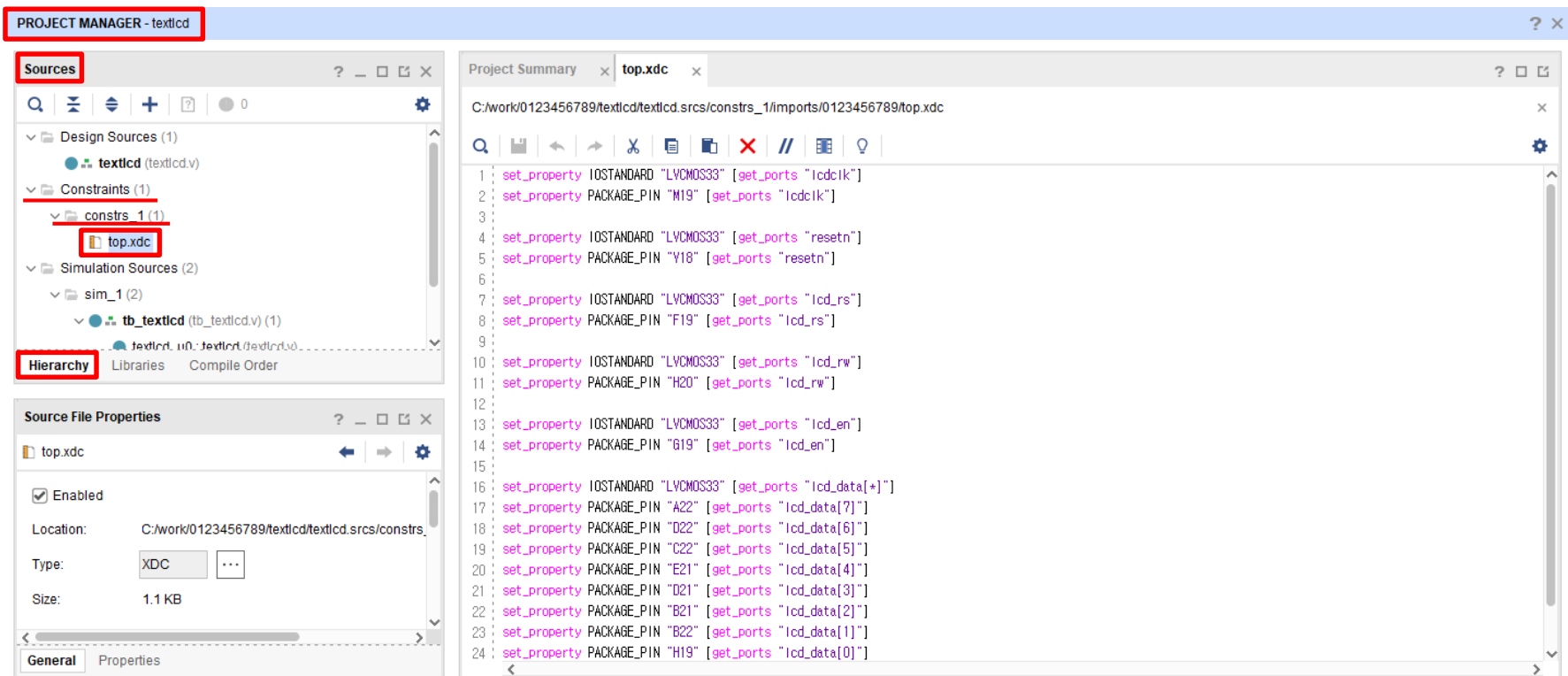
FPGA Programming

# Pin Assign (1/2)

46

□ 앞서 추가한 constraint file open

❖ Project Manager → Sources → Hierarchy → Constraints →  
constrs\_1 → top.xdc (target) double click하여 constraint file open



# Pin Assign (2/2)

47

## □ Constraint file을 통한 pin assign

- ❖ 설계한 top module(textlcd.v)의 I/O ports를 외부로 연결해야 함
- ❖ ZYNQ PL part의 pins에 top module의 I/O ports 할당
  - Constraint file(top.xdc)에 pin assign 정보 확인

```
Project Summary x top.xdc x
C:/work/0123456789/textlcd/textlcd.srsrcs/constrs_1/imports/0123456789/top.xdc

1 set_property IOSTANDARD "LVCMOS33" [get_ports "lcdclk"]
2 set_property PACKAGE_PIN "M19" [get_ports "lcdclk"]
3
4 set_property IOSTANDARD "LVCMOS33" [get_ports "resetsn"]
5 set_property PACKAGE_PIN "V18" [get_ports "resetsn"]
6
7 set_property IOSTANDARD "LVCMOS33" [get_ports "lcd_rs"]
8 set_property PACKAGE_PIN "F19" [get_ports "lcd_rs"]
9
10 set_property IOSTANDARD "LVCMOS33" [get_ports "lcd_rw"]
11 set_property PACKAGE_PIN "H20" [get_ports "lcd_rw"]
12
13 set_property IOSTANDARD "LVCMOS33" [get_ports "lcd_en"]
14 set_property PACKAGE_PIN "G19" [get_ports "lcd_en"]
15
16 set_property IOSTANDARD "LVCMOS33" [get_ports "lcd_data[*]"]
17 set_property PACKAGE_PIN "A22" [get_ports "lcd_data[7]"]
18 set_property PACKAGE_PIN "D22" [get_ports "lcd_data[6]"]
19 set_property PACKAGE_PIN "C22" [get_ports "lcd_data[5]"]
20 set_property PACKAGE_PIN "E21" [get_ports "lcd_data[4]"]
21 set_property PACKAGE_PIN "D21" [get_ports "lcd_data[3]"]
22 set_property PACKAGE_PIN "B21" [get_ports "lcd_data[2]"]
23 set_property PACKAGE_PIN "B22" [get_ports "lcd_data[1]"]
24 set_property PACKAGE_PIN "H19" [get_ports "lcd_data[0]"]
```

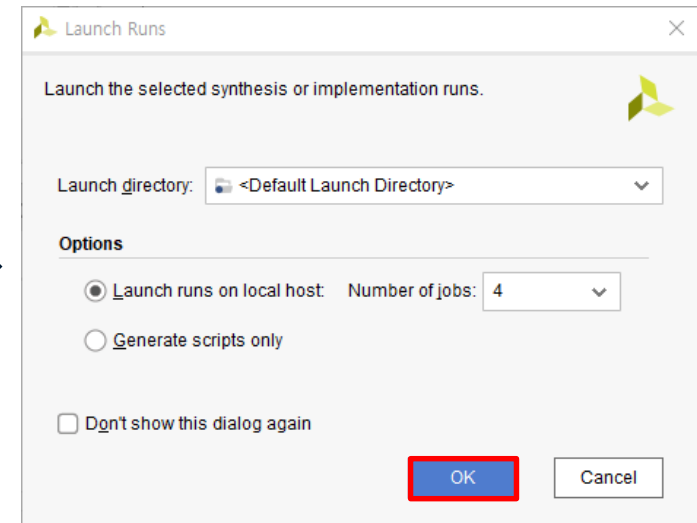
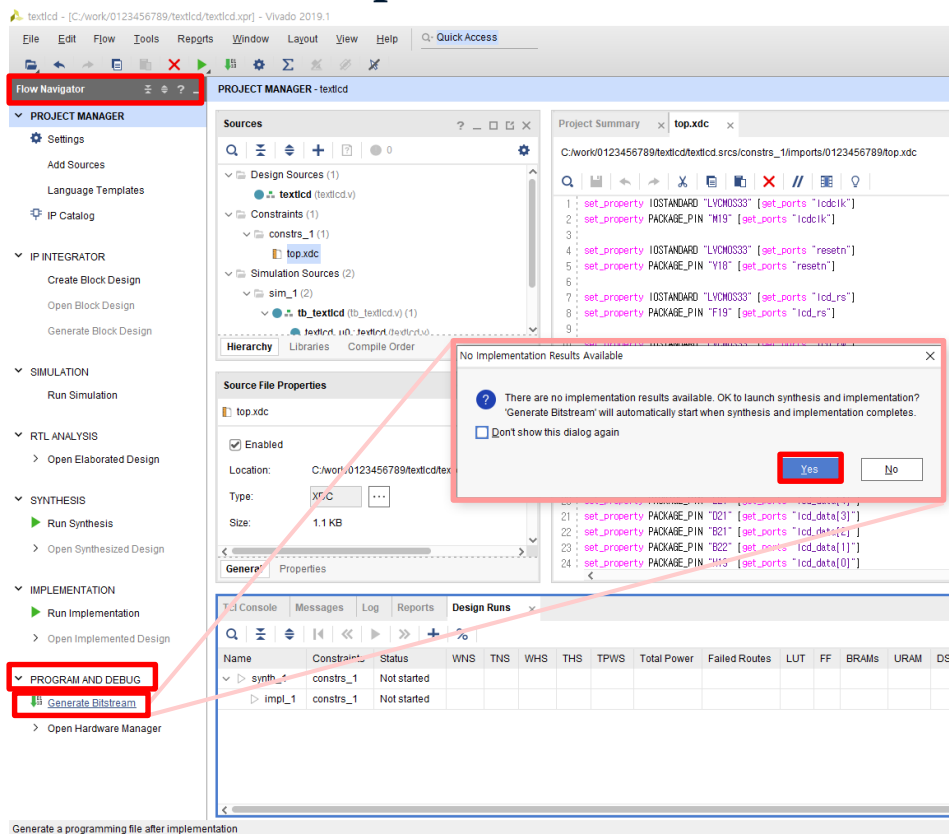
# Generate Bitstream (1/2)

48

## □ ZYNQ PL programming을 위한 bitstream file 생성

❖ Flow Navigator → Program and Debug → Generate Bitstream 클릭

❖ No Implementation Results Available window → **Yes** 클릭





# Generate Bitstream (2/2)

49

## □ Generate Bitstream 과정

❖ *Vivado 우측 상단에 status bar로 진행 과정 표시*

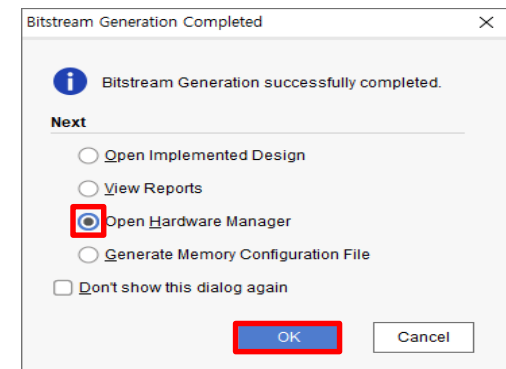
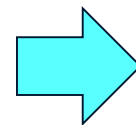
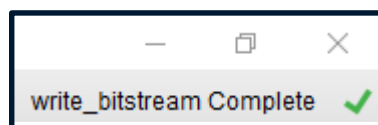
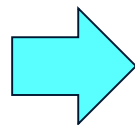
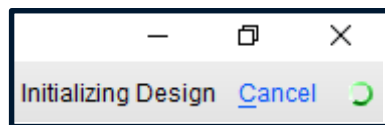
➤ 완료 후 pop-up window에서 Open Hardware Manager 체크 후 **OK** 클릭

❖ *Synthesis, implementation(place and route), generate bitstream 순으로 진행*

➤ Synthesis : HDL을 gate level의 netlist로 translate

➤ Implementation(PnR) : Netlist를 FPGA fabric에 place하고 route

➤ Generate bitstream : 최종 결과를 FPGA fabric에 programming 할 수 있는 bitstream 형태로 저장



# RPS-Z7020-TK Board Setup

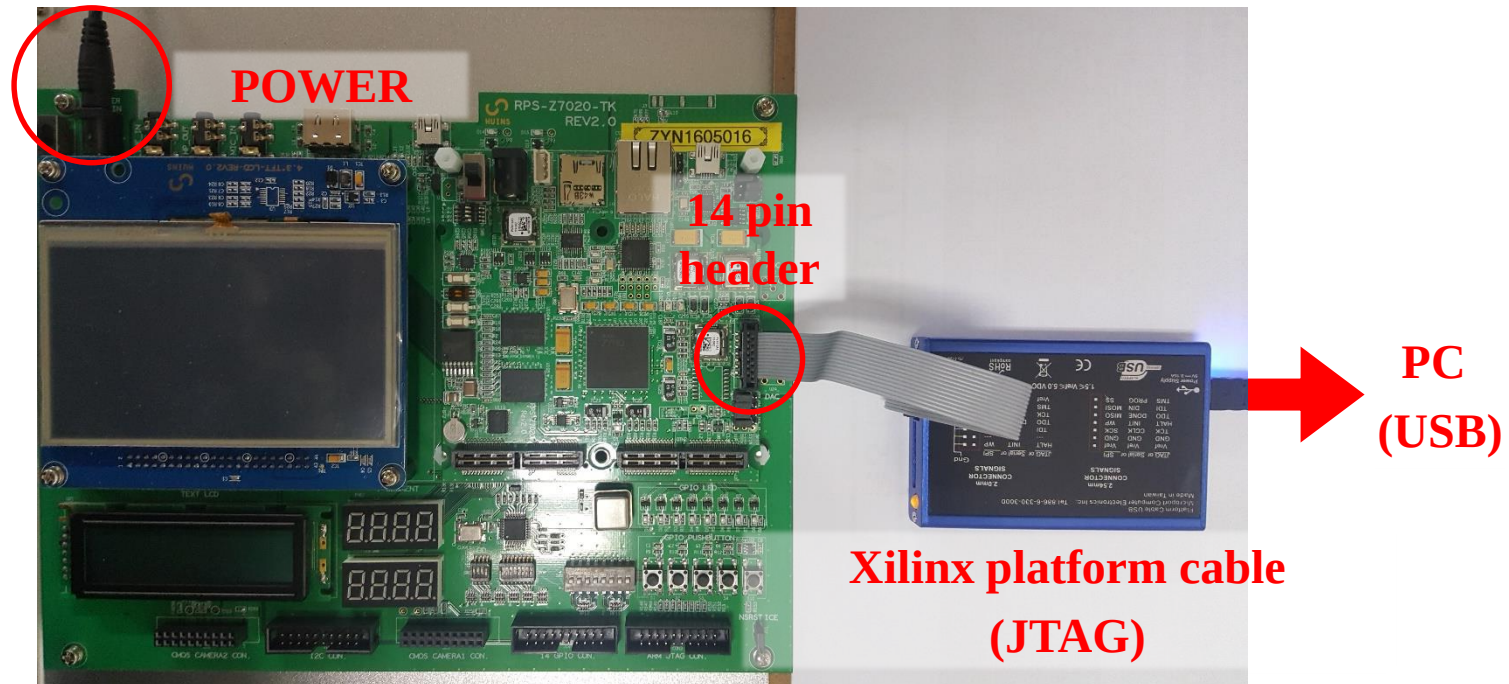
50

## ❑POWER

- ❖ 동봉되어 있는 AC adapter를 연결하여 전원 인가

## ❑Xilinx platform cable(JTAG)

- ❖ JTAG을 PC USB와 board의 14 pin header에 연결

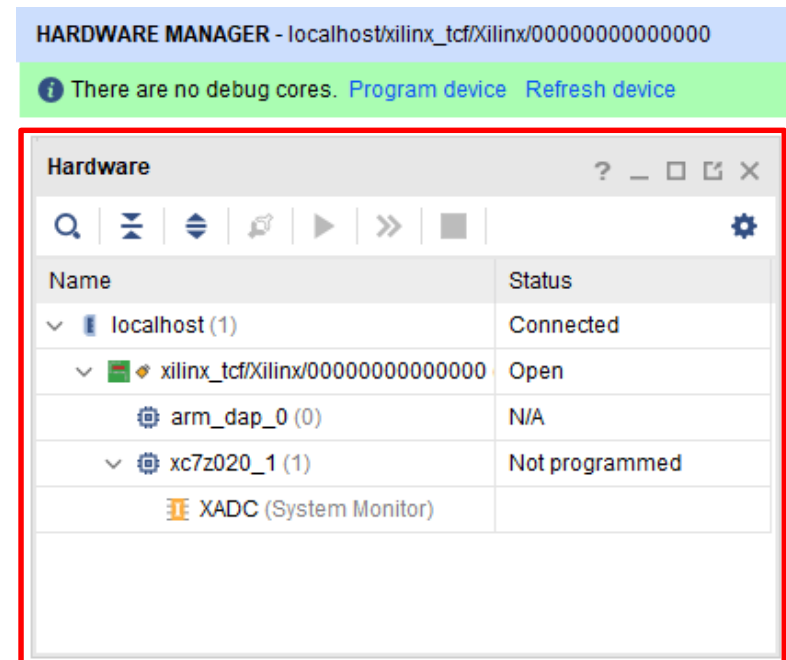
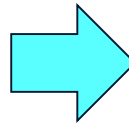
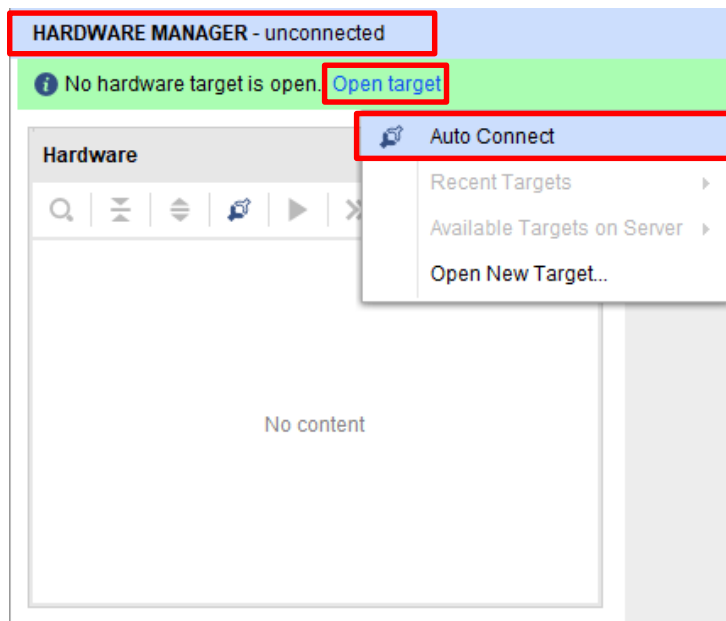


# Bitstream Programming (1/3)

51

## □ Open target

- ❖ *Open target 전 RPS-Z7020-TK board 전원 ON 필요*
- ❖ Hardware Manager window → Open target 클릭 → Auto Connect 클릭

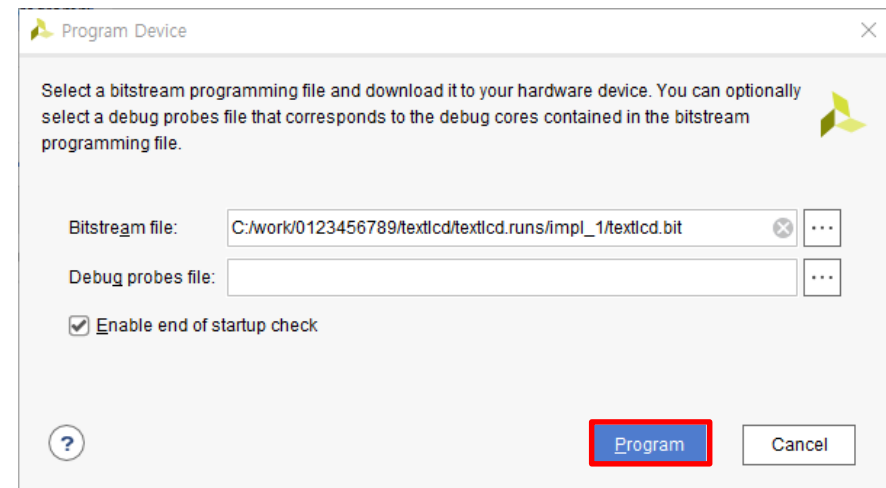
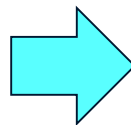
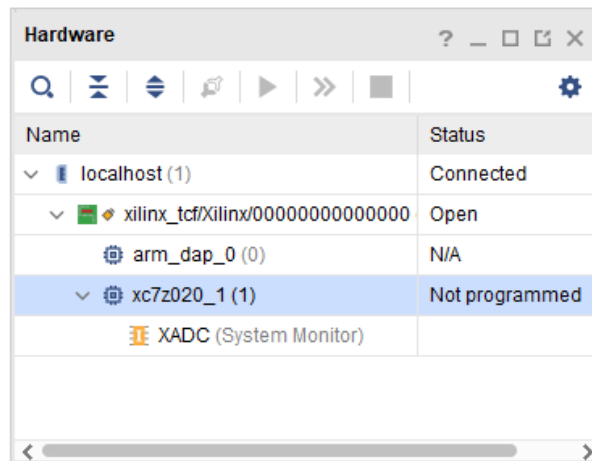
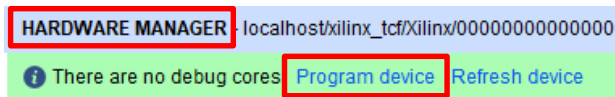


# Bitstream Programming (2/3)

52

## □ Program device

- ❖ *Bitstream file ≡ ZYNQ PL 에// programming*
- ❖ Hardware Manager window → Program device 클릭
- ❖ Program Device window에서 **Program** 클릭
  - Bitstream file을 자동으로 불러옴

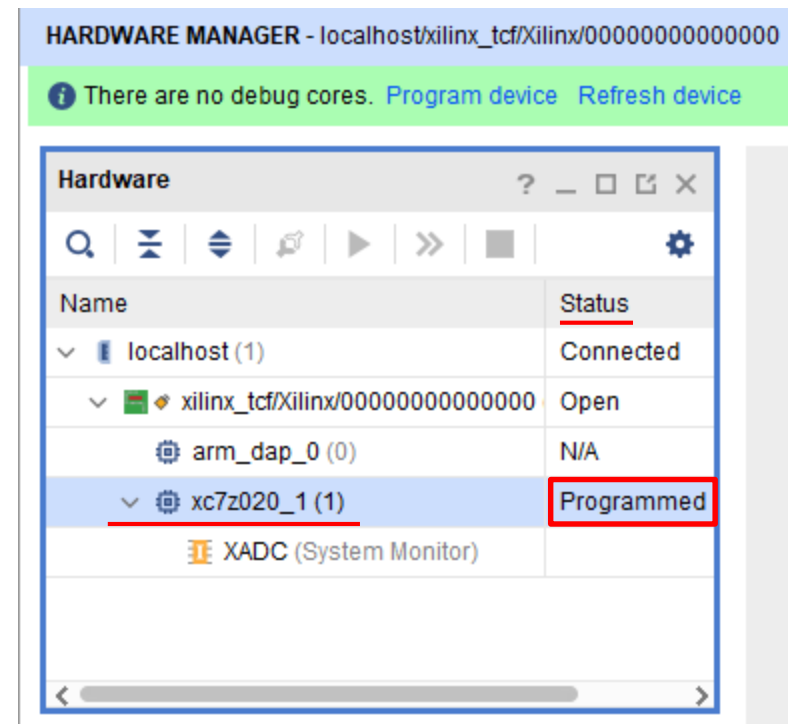
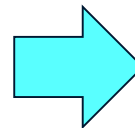


# Bitstream Programming (3/3)

53

## □ Programming 완료 확인

❖ *Bitstream file programming 완료 후 ZYNQ PL part(xc7z020\_1)의 Status가 Programmed로 변경*



## □ RPS-Z7020-TK board 확인

❖ Text LCD에 다음 message 출력 확인

Text-LCD Control  
Success SoC Lab



# Hardware Manager 종료

55

❑ *Hardware Manager*를 종료하여 *localhost server*를 닫은 후,  
*board 전원 OFF*

❖ Flow Navigator → Program and Debug에서 마우스 오른쪽 click →  
Close click → board 전원 OFF

